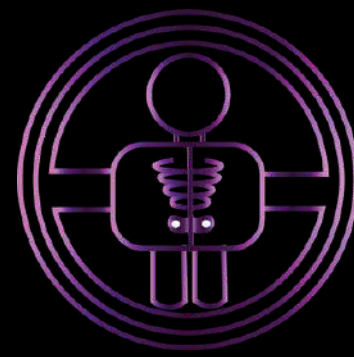




Projet de Deep Learning

Diagnostic des maladies neurologiques rares à partir des images IRM

UE Bureau d'Études Apprentissage Automatique

PRÉSENTÉ
PAR :

ASSIDAH ASMA

MASSON LOUIS

ZIRN LÉNA

21/04/2026 - M1 IMAGERIE MÉDICALE

Sommaire

Introduction

1. Réseaux pré-entraînés : VGG16

1.1. Construction du modèle

1.2. Tests

1.3. Data Augmentation

2. Analyse des résultats

2.1. Filtrés de convolution et extraction des caractéristiques

2.2. Performances du modèle

3. Évaluation du modèle et utilisation de l'API

Introduction

Contexte

Qu'est-ce qui définit une maladie rare : $1 < 2000$

Classes

- Maladie de Moya Moya
- Syndrome de Walker-Warburg
- Dystrophie musculaire congénitale de Fukuyama
- Maladie de Hallervorden-Spatz
- Pachygyrie et hypoplasie cérébelleuse

Organisation des données

Données fournies :

- Entraînement : 1400 images
- Validation : 300 images
- Test : 300 images

Pré-traitements

Images jpeg pré-traitées :

- Normalisation
- Standardisation
- Shuffle

1. Réseaux pré-entraînés : VGG16

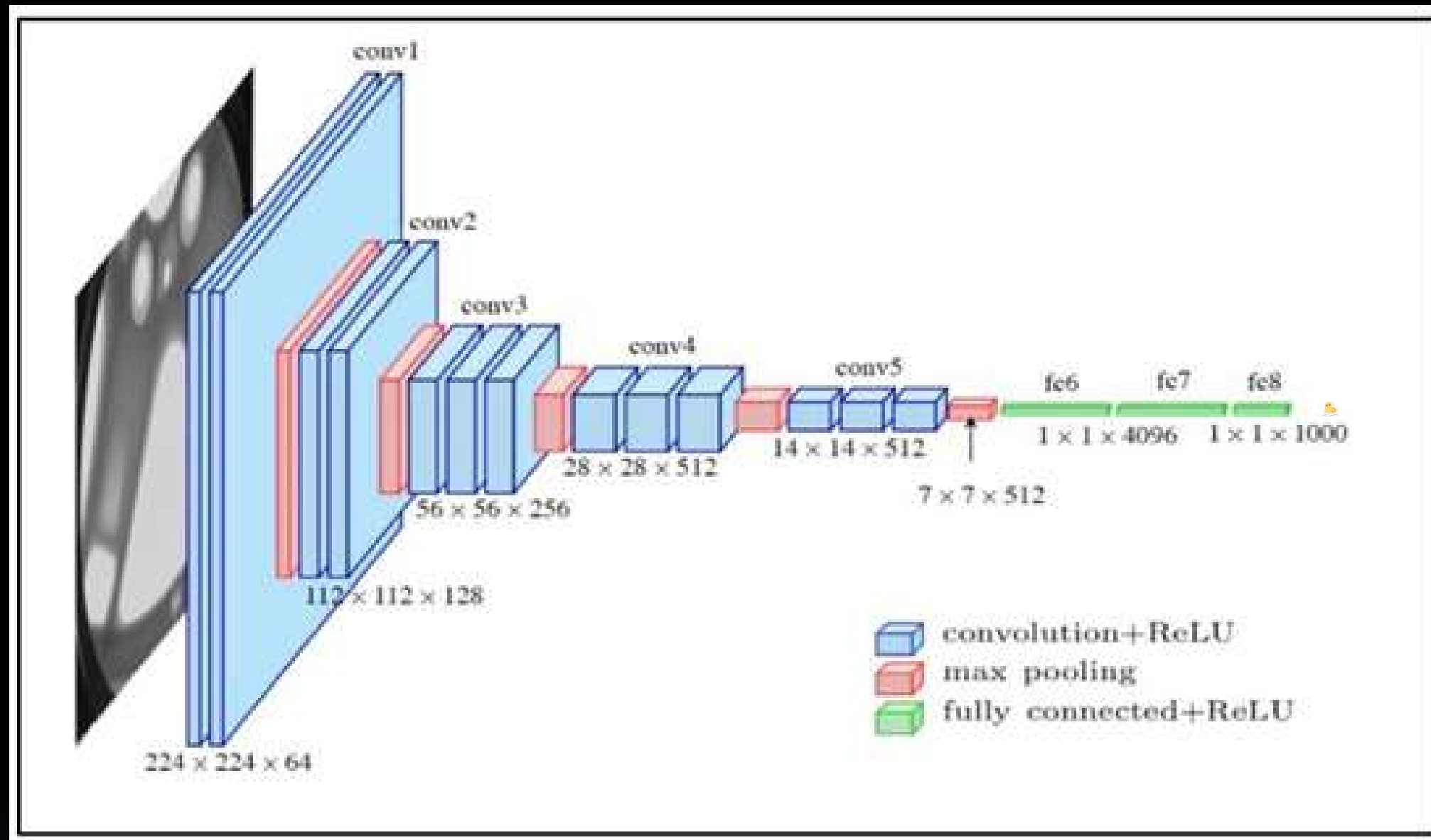


Figure 1 : Structure de VGG16 (1)

CNN basique : insuffisant

Architecture VGG16 :

- 13 couches de convolutions

- 5 couches de pooling

- 3 couches denses

1.1. VGG16 : Construction du modèle

Transfer Learning → Fine Tuning

```
# Construction du modèle : VGG16 : Fine Tuning
base_model = VGG16(include_top=False, weights='imagenet', input_shape=(224, 224, 3))
base_model.summary()

# Les paramètres des 17 premières couches sont gelés
for layer in base_model.layers[:17]:
    layer.trainable = False

# Pour voir quelles sont les couches entraînables
for i, layer in enumerate(base_model.layers):
    print(i, layer.name, layer.trainable)

model = models.Sequential([
    base_model,
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(NB_CLASSES, activation='softmax') # sortie
])

model.summary()
```

```
0 input_2 False
1 block1_conv1 False
2 block1_conv2 False
3 block1_pool False
4 block2_conv1 False
5 block2_conv2 False
6 block2_pool False
7 block3_conv1 False
8 block3_conv2 False
9 block3_conv3 False
10 block3_pool False
11 block4_conv1 False
12 block4_conv2 False
13 block4_conv3 False
14 block4_pool False
15 block5_conv1 False
16 block5_conv2 False
17 block5_conv3 True
18 block5_pool True
```

Figure 3 : 16 couches gelées de VGG16

Figure 2 : Architecture de notre modèle

1.2. VGG16 : Tests

Pour : **16** couches gelées

Numéro du modèle	neurones : couche dense1/couche dense2	Accuracy
Modèle 1	256/512	0.94999
Modèle 2	28/64	0.9733
Modèle 3	256/128	0.98

Pour :

1e couche dense : **256** neurones

2e couche dense : **128** neurones

Numéro du modèle	Nombre de couches gelées	Accuracy
Modèle 4	14	0.2
Modèle 5	16	0.98
Modèle 6	15	0.9599

Modèle retenu :

-16 couches gelées

-1e couche dense: 256 neurones

-2e couche dense : 128 neurones

1.3. VGG16 : Data Augmentation

Modèle 7

```
# 1e essai :

datagen_train = ImageDataGenerator(
    rescale = 1./ 255,
    width_shift_range=0.05,
    height_shift_range=0.05,
    horizontal_flip=True,
    rotation_range=2,
    brightness_range=[0.9, 1.1],
    zoom_range=[0.9, 1.1]
)

# accuracy : 0.98333
```

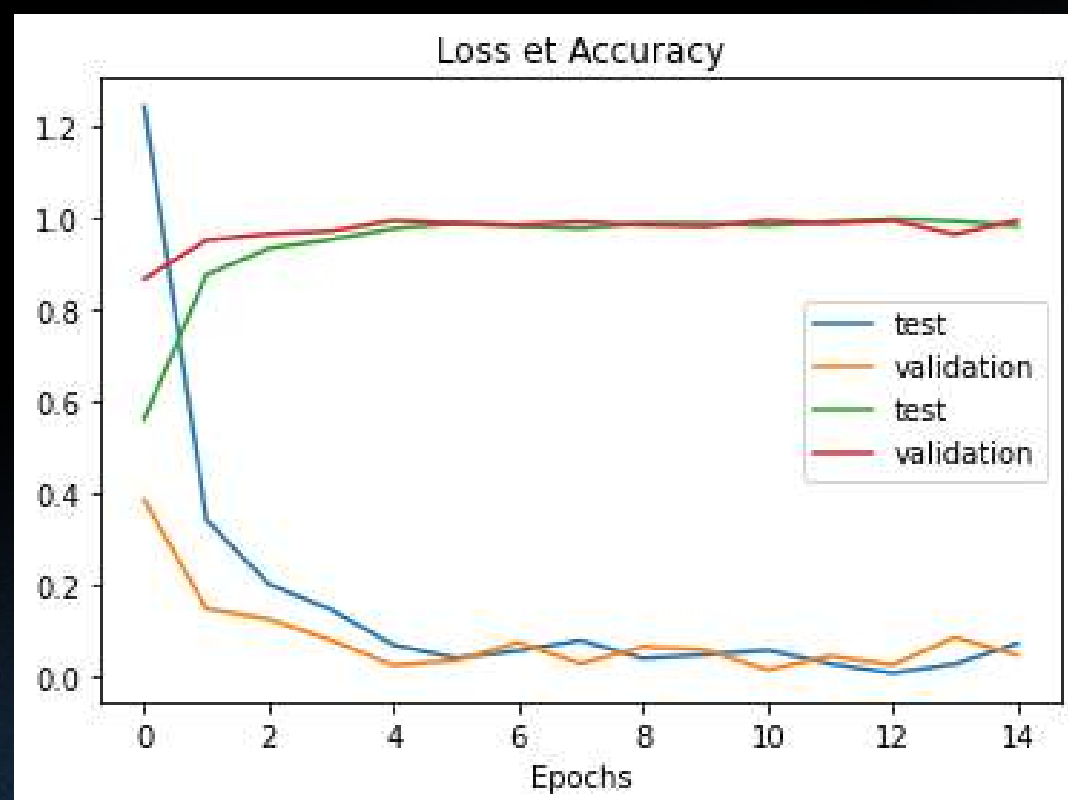


Figure 4 : Loss et Accuracy modèle 7

Modèle 8

```
# 2e essai :

datagen_train = ImageDataGenerator(
    rescale = 1./ 255,
    width_shift_range=0.02,
    height_shift_range=0.02,
    horizontal_flip=True,
    rotation_range=2,
    brightness_range=[0.3, 1.7],
    zoom_range=[0.9, 1.1]
)

# accuracy : 0.97
```

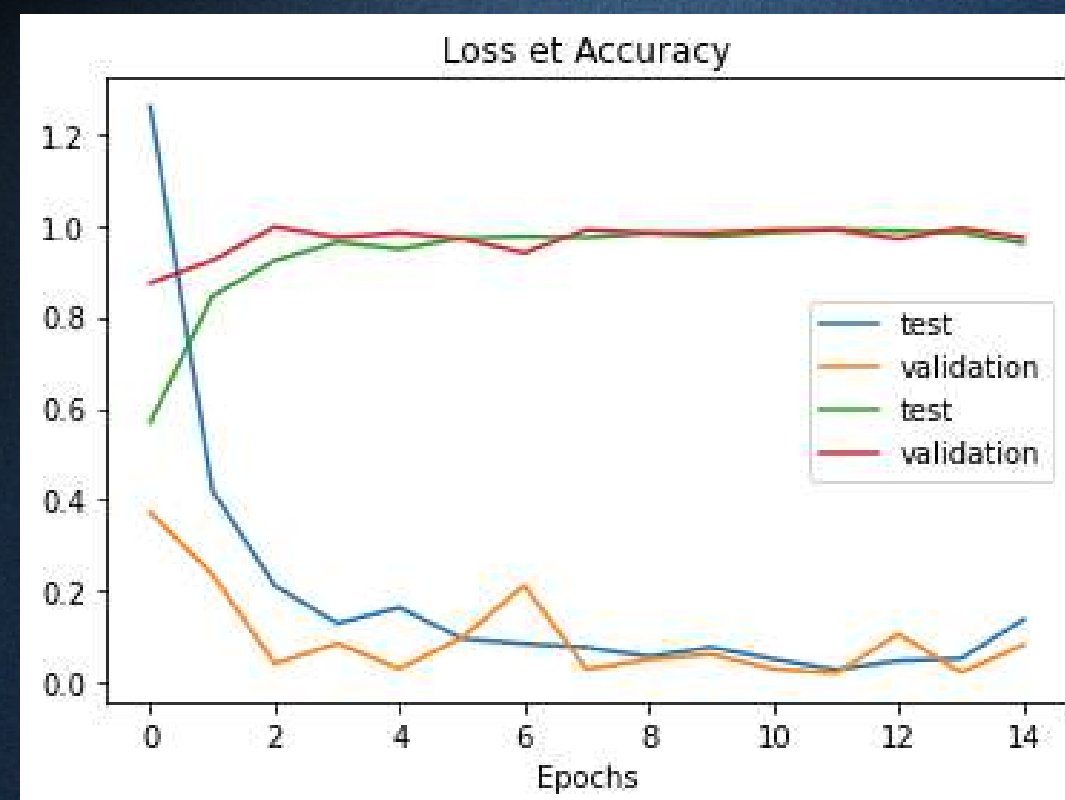


Figure 5 : Loss et Accuracy modèle 8

Modèle 9

```
# 3e essai :

datagen_train = ImageDataGenerator(
    rescale = 1./ 255,
    width_shift_range=0.02,
    height_shift_range=0.02,
    horizontal_flip=True,
    rotation_range=2,
    # brightness_range=[0.3, 1.7],
    zoom_range=[0.7, 1.3]
)

# accuracy : 0.99
```

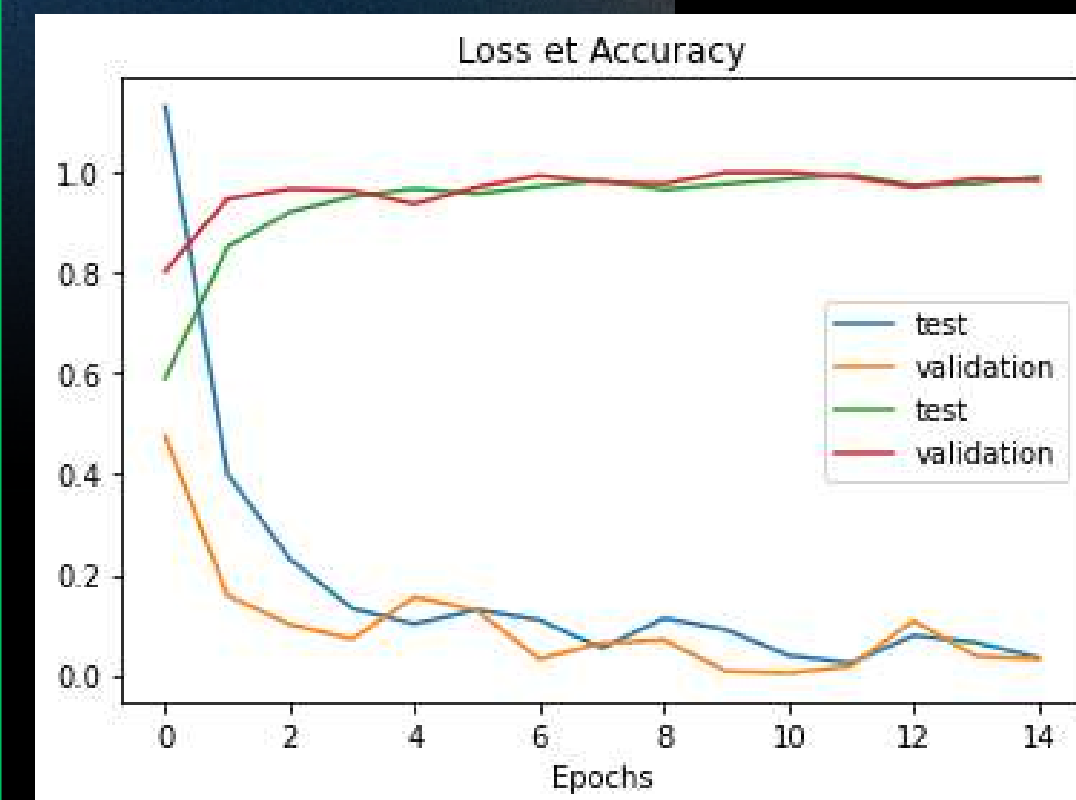


Figure 6 : Loss et Accuracy modèle 9

1.3. VGG16 : Data Augmentation

Modèle final

Model: "sequential_7"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 7, 7, 512)	14714688
flatten_7 (Flatten)	(None, 25088)	0
dense_21 (Dense)	(None, 256)	6422784
dropout_14 (Dropout)	(None, 256)	0
dense_22 (Dense)	(None, 128)	32896
dropout_15 (Dropout)	(None, 128)	0
dense_23 (Dense)	(None, 5)	645

Total params: 21,171,013
Trainable params: 8,816,133
Non-trainable params: 12,354,880

Figure 7 : Architecture de notre modèle

```
# 3e essai :  
  
datagen_train = ImageDataGenerator(  
    rescale = 1./ 255,  
    width_shift_range=0.02,  
    height_shift_range=0.02,  
    horizontal_flip=True,  
    rotation_range=2,  
    # brightness_range=[0.3, 1.7],  
    zoom_range=[0.7, 1.3]  
)  
  
# accuracy : 0.99
```

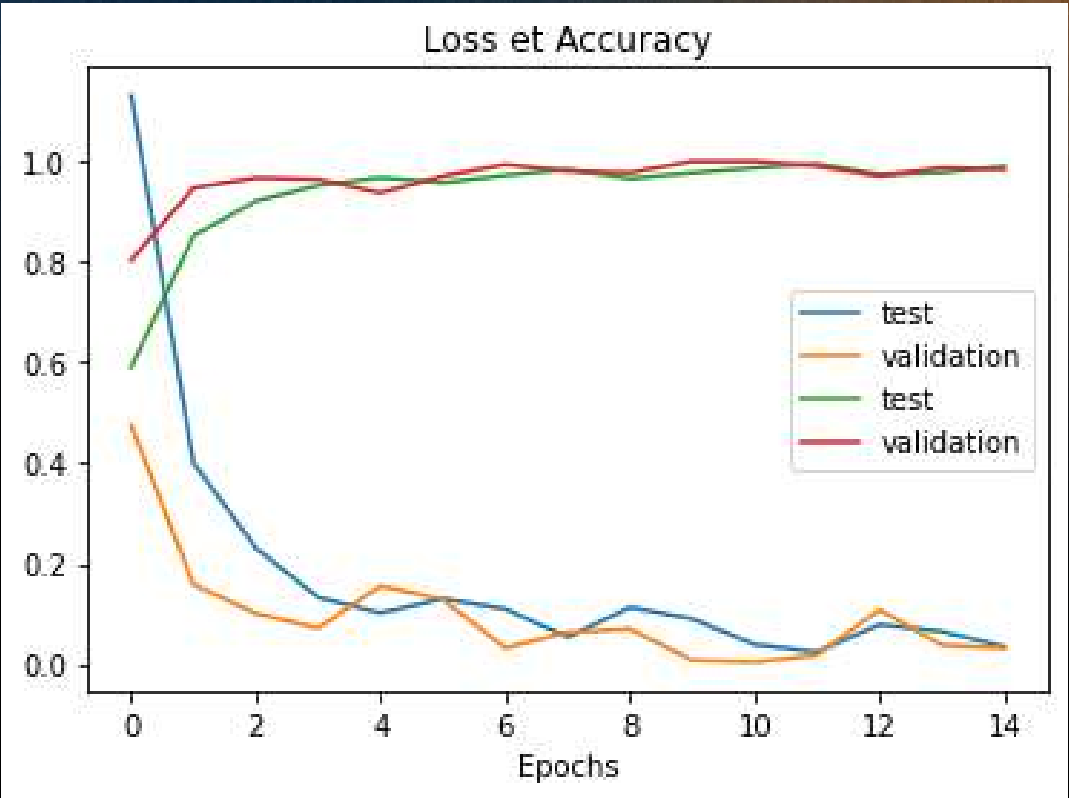


Figure 6 : Loss et Accuracy modèle 9

2.1. Analyses des résultats : Filtres et caractéristiques

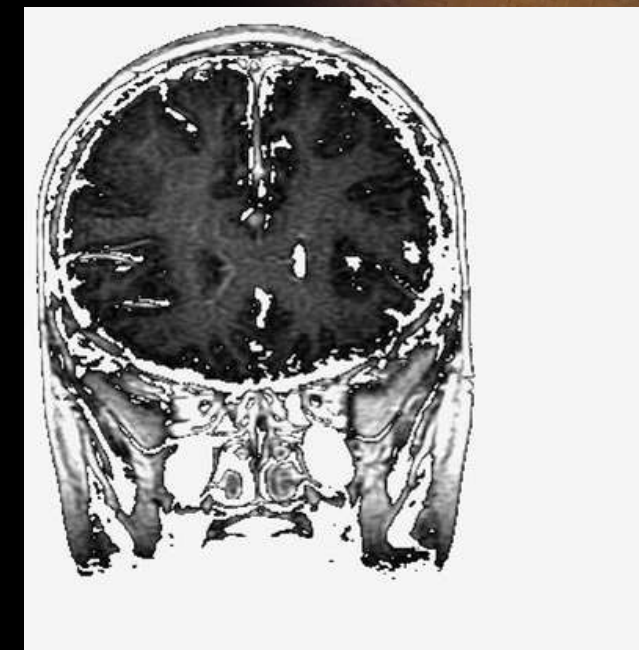
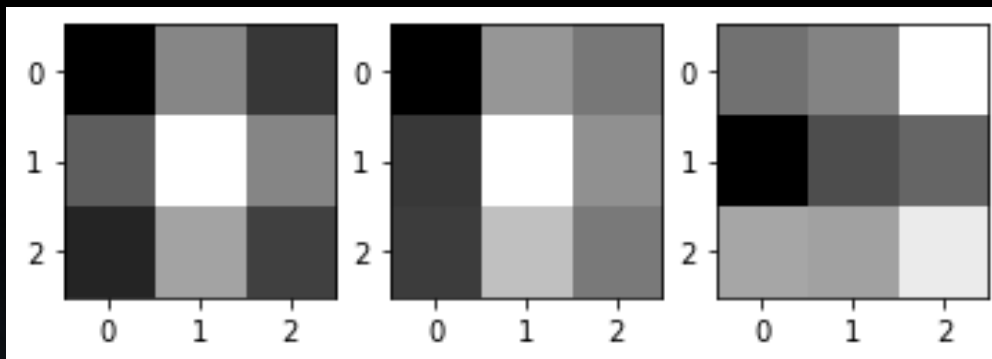
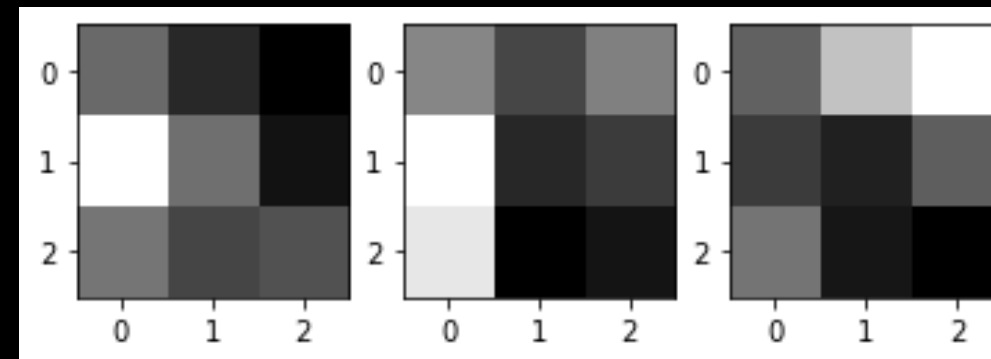


Figure 7 : Image
d'origine / Maladie de
Hallervorden-Spatz

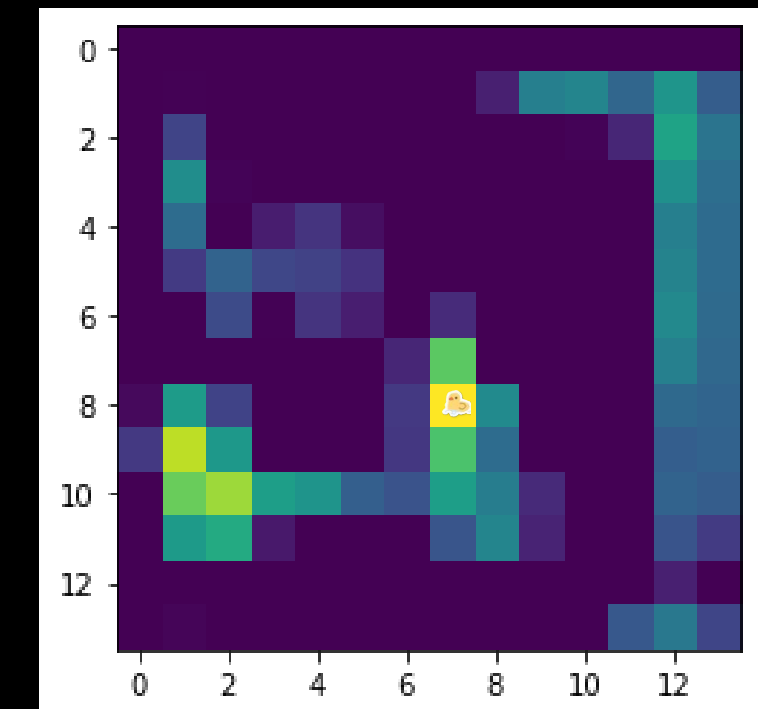
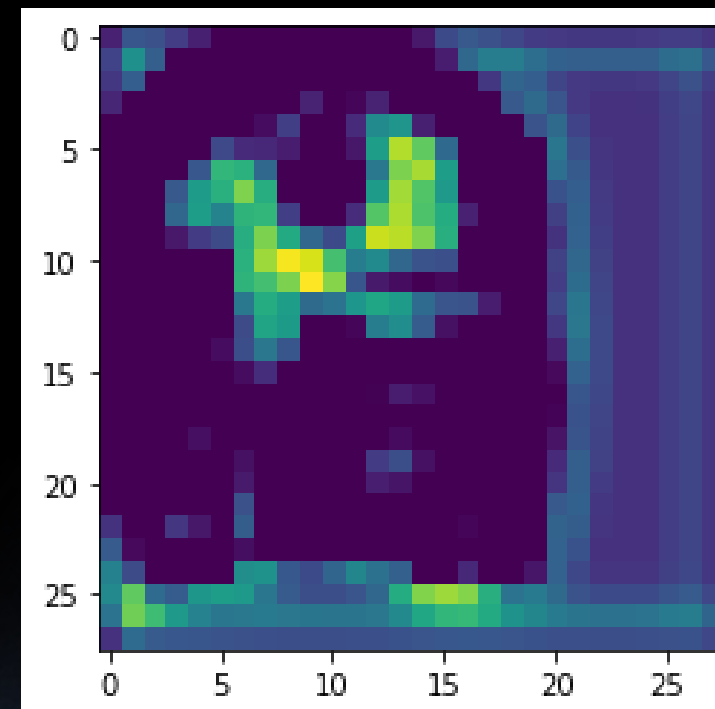
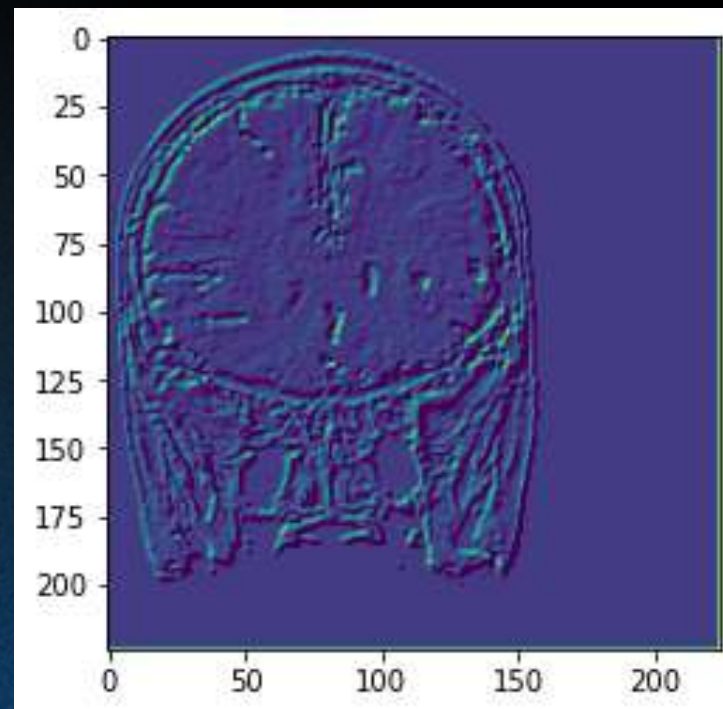
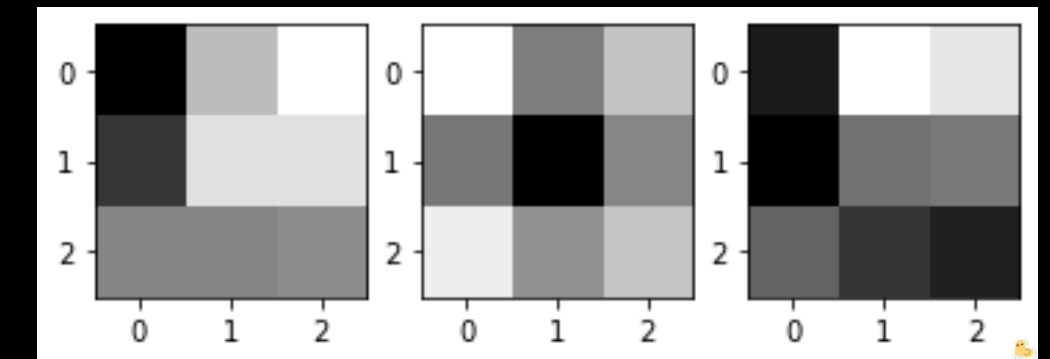
Couche 1



Couche 11



Couche 16



2.2. Analyses des résultats : Performances du modèle

Accuracy du modèle: 0.99

Matrice de confusion

Classe 0	[60	0	0	0	0]
Classe 1	[0	59	1	0	0]
Classe 2	[0	0	60	0	0]
Classe 3	[0	0	0	60	0]
Classe 4	[0	0	0	0	60]

Found 300 images belonging to 5 classes

	precision	recall	f1-score	support
0	1.00	1.00	1.00	60
1	1.00	0.98	0.99	60
2	0.98	1.00	0.99	60
3	1.00	1.00	1.00	60
4	1.00	1.00	1.00	60
accuracy			1.00	300
macro avg	1.00	1.00	1.00	300
weighted avg	1.00	1.00	1.00	300

3. Évaluation du modèle

Présentation

Exploitation du système via Streamlit sur une interface web

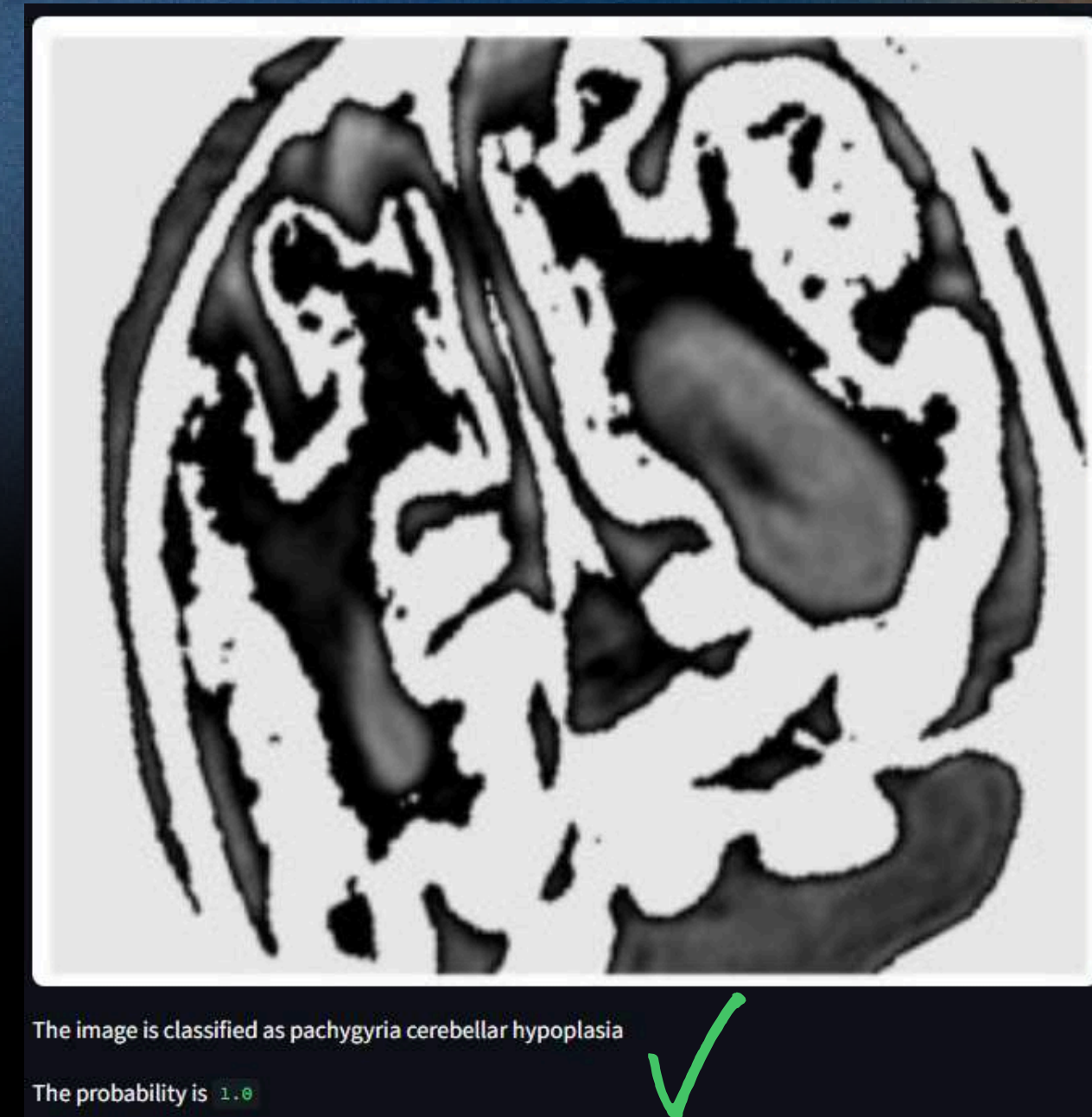
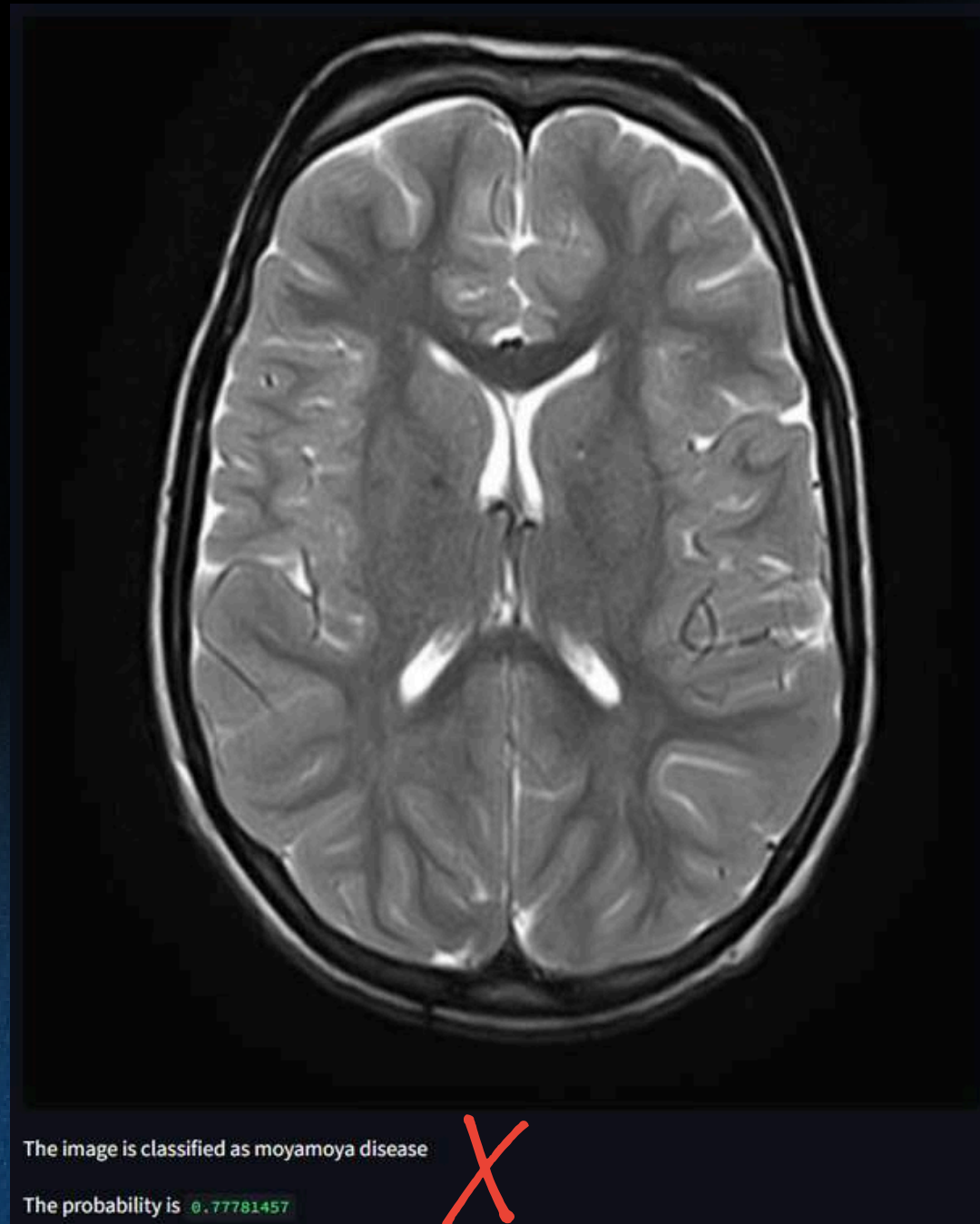
Étapes

- Chargement du modèle
- Dépôt de l'image
- Prétraitement automatique
- Prédiction et affichage

Objectifs

Démontrer robustesse et limites

3. Évaluation du modèle

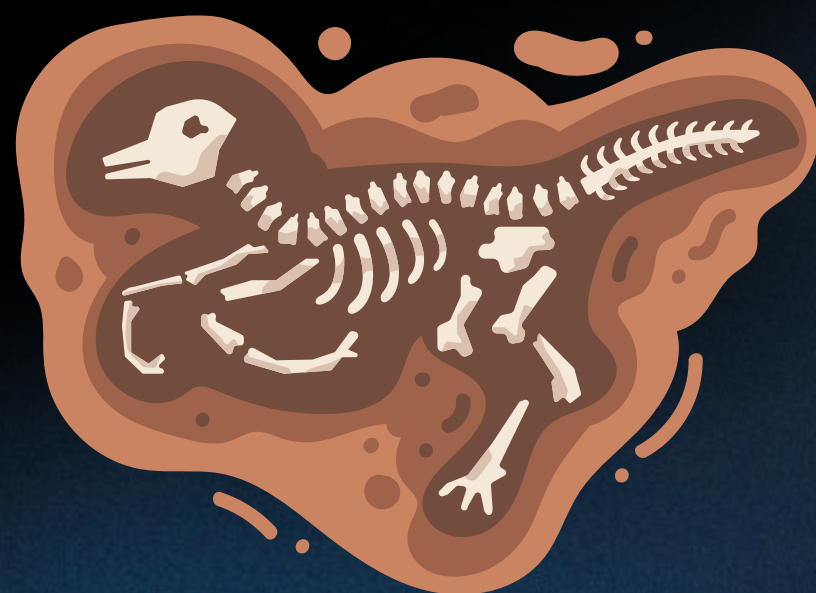


Objectifs

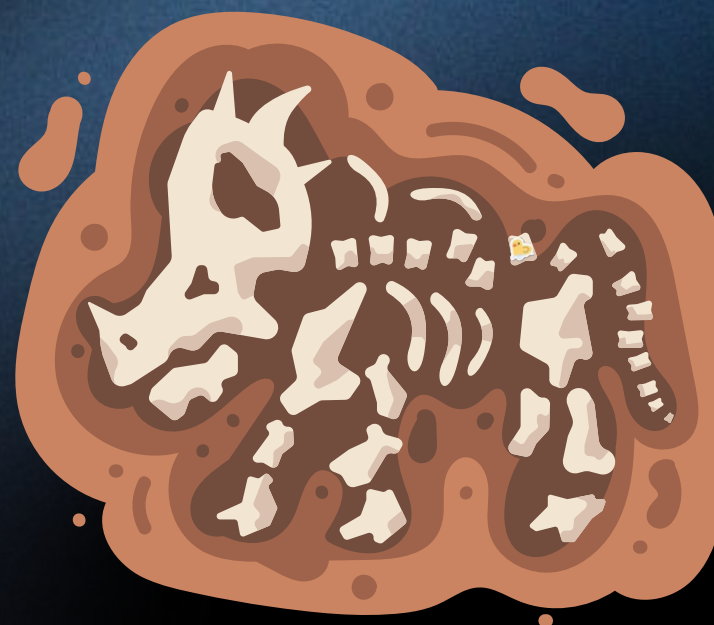
Démontrer robustesse et limites

Conclusion

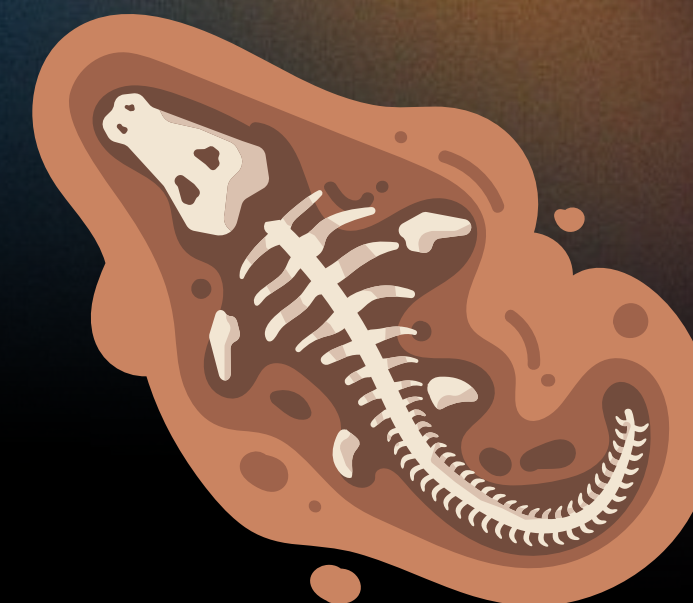
MERCI DE VOTRE ATTENTION



Email
assidah.asma@gmail.com



Email
louis.masson2612@gmail.com



Email
lenazirn638@gmail.com

Actuellement en recherche de stage voici nos mails

- [1] A. Hanzala, M. Sajjad, T. Akter, H. Kaur, and M. S. Rahman, 'NeuroFusion: A forensic enriched ensemble framework for cerebellum disease classification', *Neuroscience Informatics*, vol. 6, no. 1, p. 100251, Mar. 2026, doi: 10.1016/j.neuri.2025.100251.
- [2] 'VGGNet-16 Architecture: A Complete Guide'. Accessed: Apr. 20, 2026. [Online]. Available: <https://kaggle.com/code/blurredmachine/vggnet-16-architecture-a-complete-guide>
- [3] S. Hosseini, 'automatique (machine learning)'.
- [4] R. Thibodeau, 'Creutzfeldt-Jakob disease | Radiology Case | Radiopaedia.org', Radiopaedia. Accessed: Apr. 19, 2026. [Online]. Available: <https://radiopaedia.org/cases/creutzfeldt-jakob-disease-20>
- [5] S. P. Ganta, 'Fukuyama congenital muscular dystrophy | Radiology Case | Radiopaedia.org', Radiopaedia. Accessed: Apr. 19, 2026. [Online]. Available: <https://radiopaedia.org/cases/fukuyama-congenital-muscular-dystrophy>
- [6] B. D. Muzio, 'Hemolytic uremic syndrome | Radiology Reference Article | Radiopaedia.org', Radiopaedia. Accessed: Apr. 19, 2026. [Online]. Available: <https://radiopaedia.org/articles/haemolytic-uraemic-syndrome>
- [7] Keras: Deep Learning for humans'. Accessed: Apr. 19, 2026. [Online]. Available: <https://keras.io/>
- [8] S. Aslam, H. H. Mughal, N. Akram, I. Maqbool, A. Shafi, and Md. A. Haque, 'An insight to Fukuyama congenital muscular dystrophy (FCMD): Report of a rare case', *Medical Reports*, vol. 3, p. 100028, Feb. 2024, doi: 10.1016/j.hmedic.2023.100028.
- [9] A. Younis, L. Qiang, C. O. Nyatega, M. J. Adamu, and H. B. Kawuwa, 'Brain Tumor Analysis Using Deep Learning and VGG-16 Ensembling Learning Approaches', *Applied Sciences*, vol. 12, no. 14, p. 7282, Jan. 2022, doi: 10.3390/app12147282.